MODULE 5 | MACHINE PROBLEMS

Computer Graphics and Visual Computing • C++ / OpenGL / GLUT • Vertex Buffer Objects and Indexed Rendering

Name: _Maria Andrea V. Malonzo_____ Section: ____TN35_ Date: ____2026-10-22__

Name: PAMPILON, Patrick_____ Section: ___TN35_ Date: __2026-10-22_____

General Instructions

- Complete all three scenario-based machine problems independently. Submit source code, screenshots of working outputs, and written answers to the open-ended questions.
- Use C++, OpenGL and GLUT in a compatibility-capable context. Use VBOs for vertex positions; use an element array buffer where indexed rendering is required. GLEW is applicable to the Windows/MSYS2 environment described in the module.
- Every program must accept user input and present a menu. A console menu may control the graphics window; after each valid choice, update the display. Validate inputs and provide an Exit option.
- Initialize buffers once where possible, reuse them during drawing, and delete them during orderly shutdown. Use glDrawArrays with the actual vertex count; GL_NORMAL_ARRAY is not needed for index buffers.

MACHINE PROBLEM 01

Campus Traffic Signal Visualizer

Beginner • VBO + glDrawArrays()

Scenario

You have been assigned to create a prototype for a campus road-safety orientation. The security office needs a simple traffic-signal visualizer that can display different signal states and let an operator change the scene during a demonstration.

Problem Statement

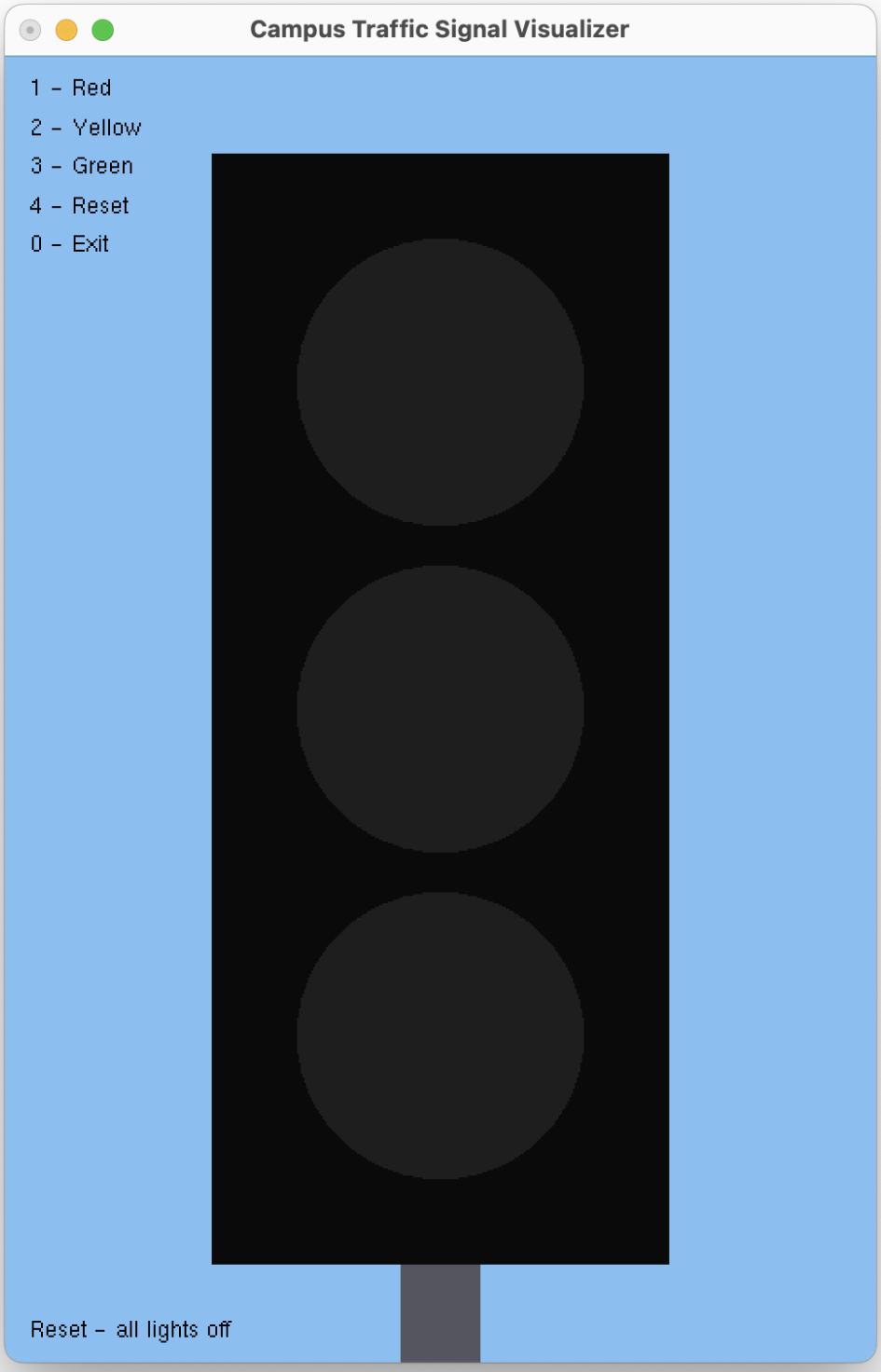
Develop a menu-driven C++ OpenGL application that draws a traffic-light housing and three geometric signal indicators. The program must store geometry positions in at least one Vertex Buffer Object (VBO) and render its primitives using glDrawArrays(). The operator enters a selection to switch the active signal or reset the display.

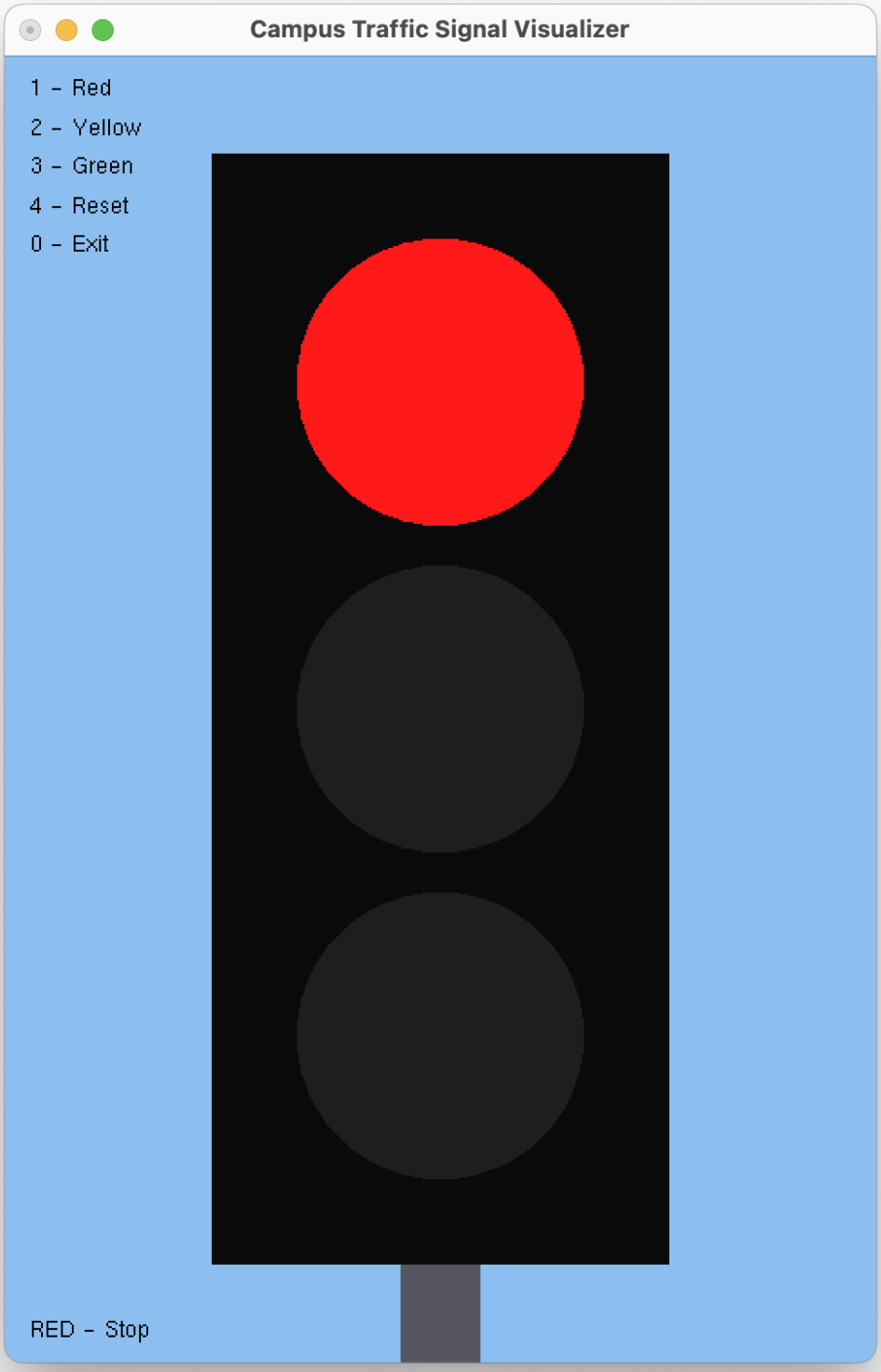
Functional and Technical Requirements

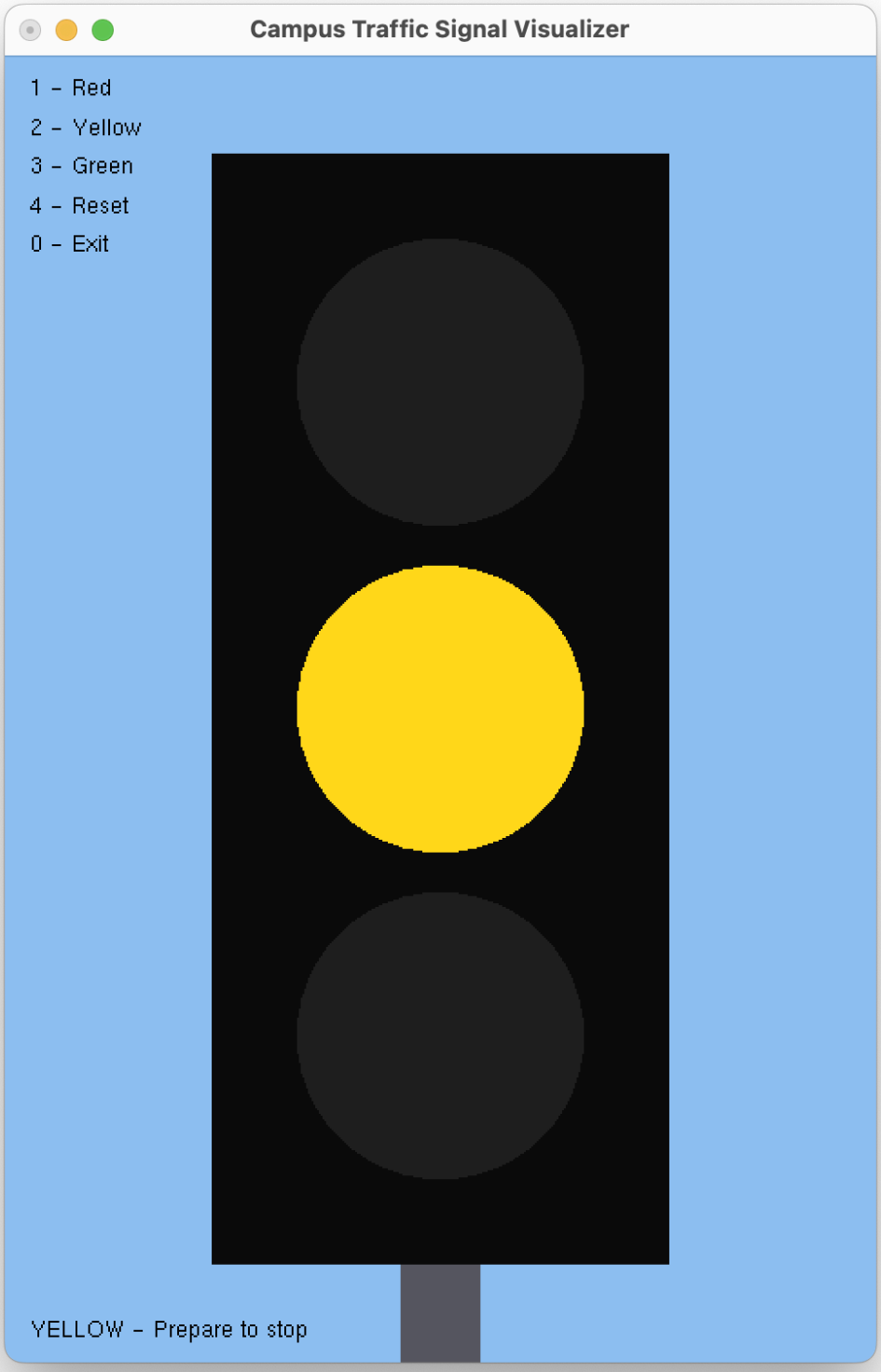
1. Draw a traffic-light body and three vertically arranged signal shapes using triangles (a circle-like indicator may be approximated using several triangles).
2. Use at least one VBO to store vertex positions; do not use glBegin()/glEnd() for the required signal geometry.
3. Provide menu choices: 1 Red / Stop, 2 Yellow / Caution, 3 Green / Go, 4 All Off, 5 Reset, 0 Exit.
4. Accept a numeric menu selection, reject invalid values, and update the graphics window after valid input.
5. Display the active state visibly through color; keep inactive indicators dark.

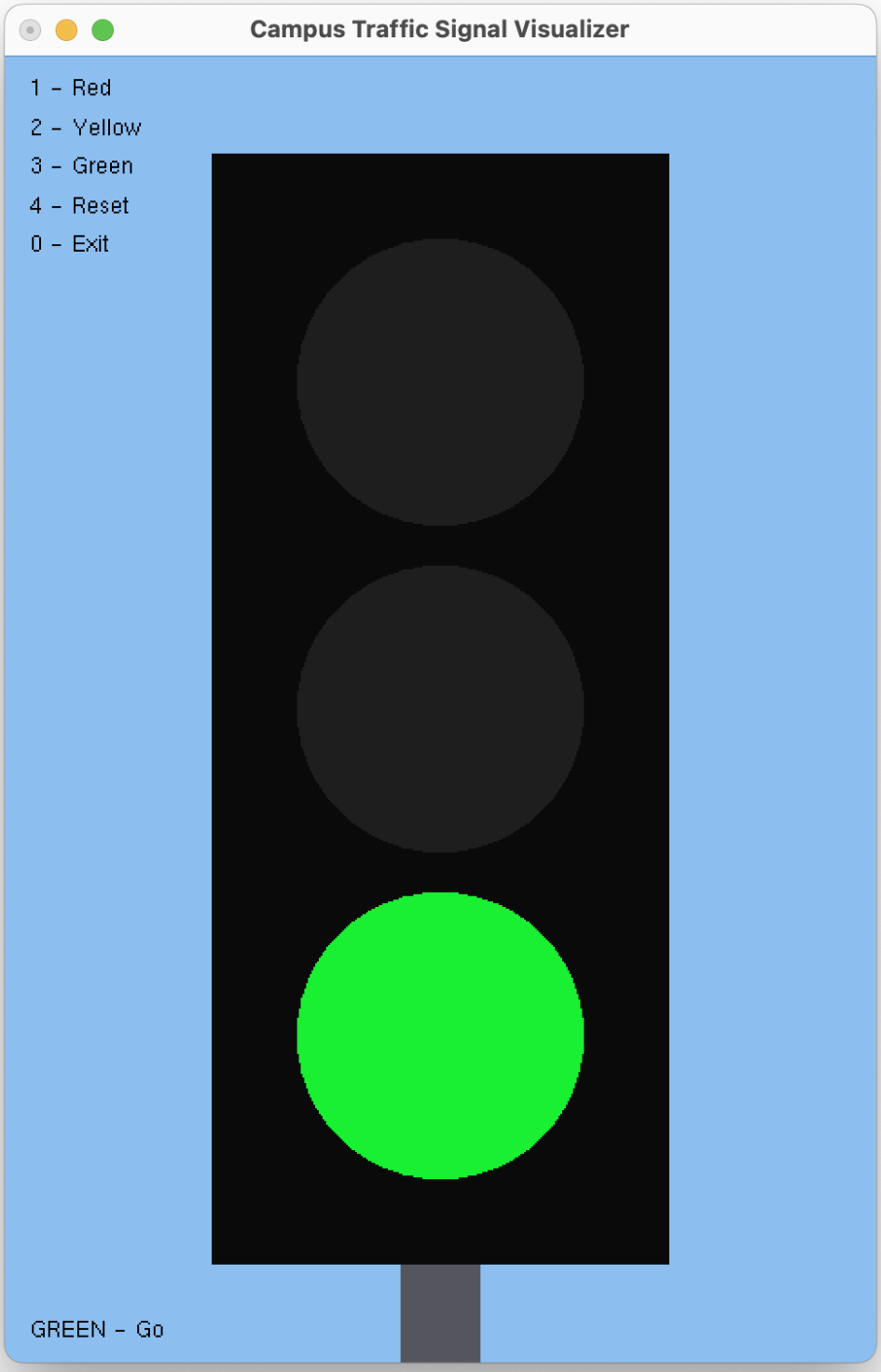
Expected Output

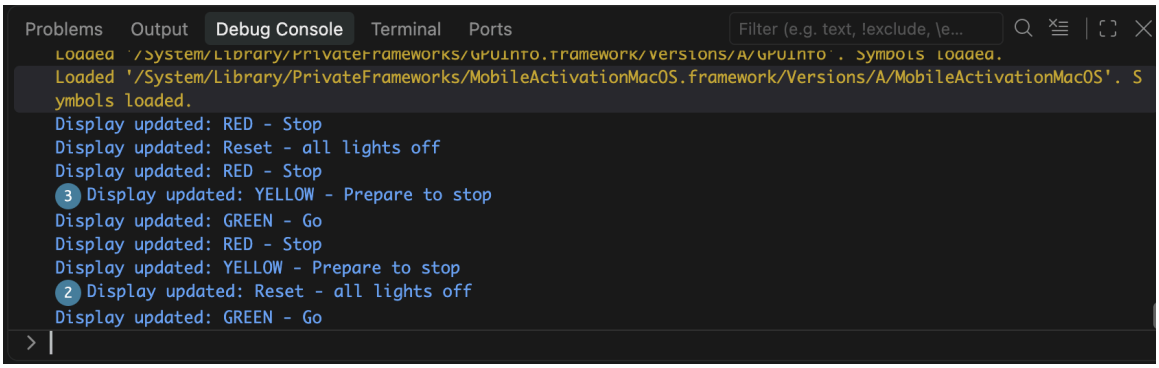
A functioning graphical window that responds to the operator's menu selections, with clear visual changes and no crashes on invalid input.


Campus Traffic Signal Visualizer
1 – Red
2 – Yellow
3 – Green
4 – Reset
0 – Exit
Reset – all lights off


Campus Traffic Signal Visualizer
1 - Red
2 - Yellow
3 - Green
4 - Reset
0 - Exit
RED - Stop


Campus Traffic Signal Visualizer
1 - Red
2 - Yellow
3 - Green
4 - Reset
0 - Exit
YELLOW - Prepare to stop


Campus Traffic Signal Visualizer
1 - Red
2 - Yellow
3 - Green
4 - Reset
0 - Exit
GREEN - Go



Open-Ended Questions

1. Explain how a VBO differs from specifying each vertex through glVertex2f().

`glVertex2f()` uses immediate mode, sending individual vertex coordinates from the CPU to the GPU one by one every frame, which creates a huge processing bottleneck. A VBO (Vertex Buffer Object) instead uploads all vertex data into high-speed GPU memory just once during setup. During rendering, the program simply binds the VBO and draws all shapes at once using a single batch command. This drastically reduces CPU overhead and improves overall frame rates.

2. Why is GL_STATIC_DRAW a reasonable initial choice for this design? Under what change would GL_DYNAMIC_DRAW be more appropriate?

`GL_STATIC_DRAW` is a great initial choice because the physical shape and positions of the traffic light geometry never change while the program runs. The GPU can optimize storage knowing this vertex data will be set once and drawn many times without modifications. If the geometry needed to move, stretch, or update its vertex positions dynamically every frame, like an animated or resizable signal, `GL_DYNAMIC_DRAW` would be much more appropriate. This tells the GPU driver to prepare the memory buffer for frequent updates and re-uploads.

3. How would you redesign your geometry to support a traffic light with a different number of indicators?

I would start by replacing hardcoded constants with a dynamic array or vector to store light properties like Y-positions and vertex offsets. Then, I would update `buildGeometry()` to calculate the height of the housing and circle positions dynamically based on how many lights are needed. Finally, I would update the drawing loop in `display()` to iterate through the total number of lights instead of stopping at three. This allows the code to draw any number of lamps without hardcoding their positions.

4. What problems might occur if the program recreated and deleted the VBO every frame?

Recreating and deleting the VBO on every frame creates heavy driver overhead and severe performance lag. Constantly allocating and freeing GPU memory causes pipeline synchronization stalls that ruin the frame rate. It can also lead to memory fragmentation in

VRAM over time. If any buffer isn't freed properly due to a bug, it will quickly cause a VRAM memory leak and crash the program.

Submission Checklist

/☐ C++ source file(s) /☐ Output screenshots /☐ Menu/input evidence ☐ Written answers

MACHINE PROBLEM 02

Community Park Layout Planner

Intermediate • Indexed geometry + glDrawElements()

Scenario

A local planning committee wants an interactive prototype for a small community park. They need to compare layouts for a walking path, central activity area, and garden plots without manually redrawing the design.

Problem Statement

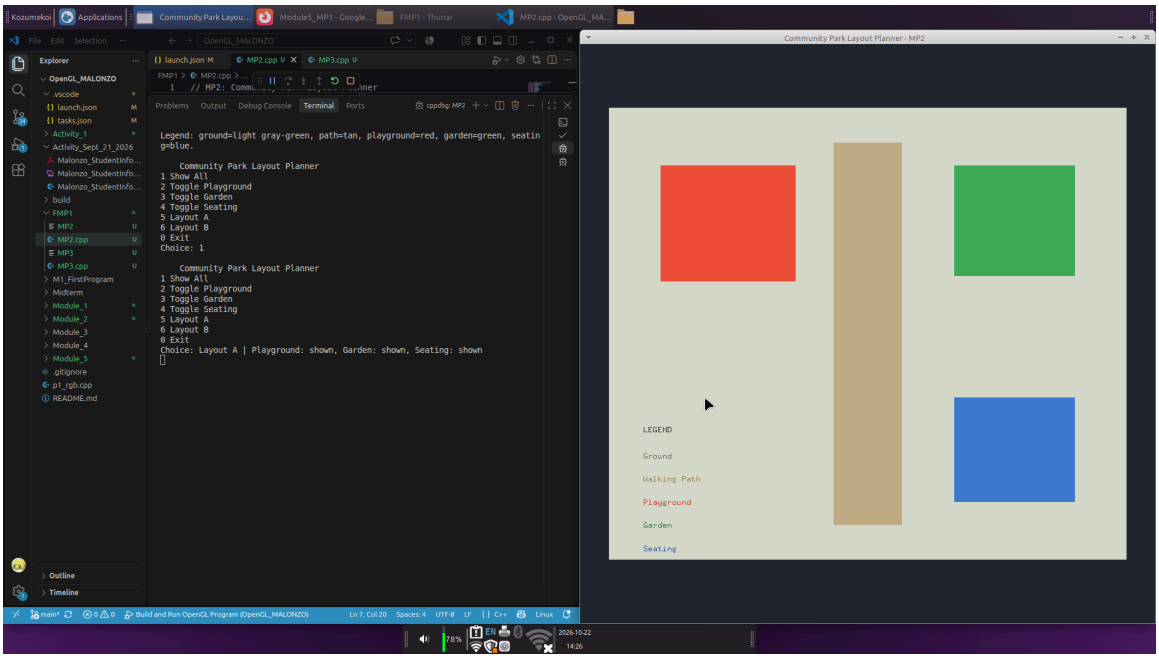
Create a menu-driven 2D park layout viewer. Represent rectangular features using two triangles each. Store unique vertex positions in a VBO and their triangle indices in an element array buffer; render the indexed geometry using glDrawElements(). The user chooses which feature to show or hide and can select a layout preset.

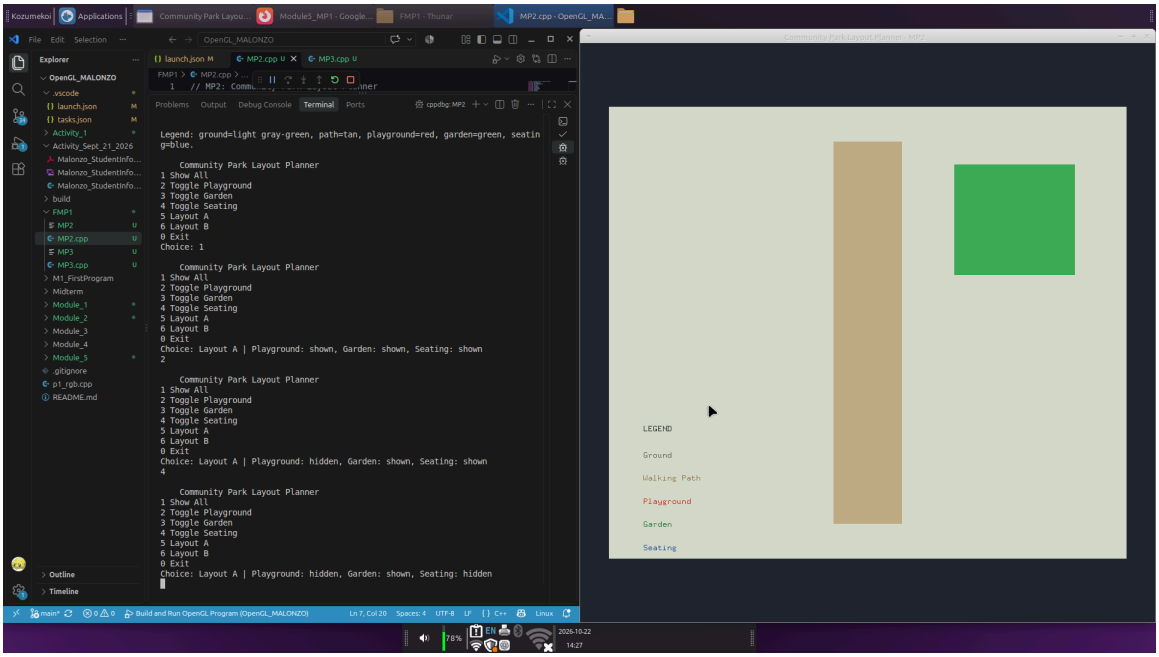
Functional and Technical Requirements

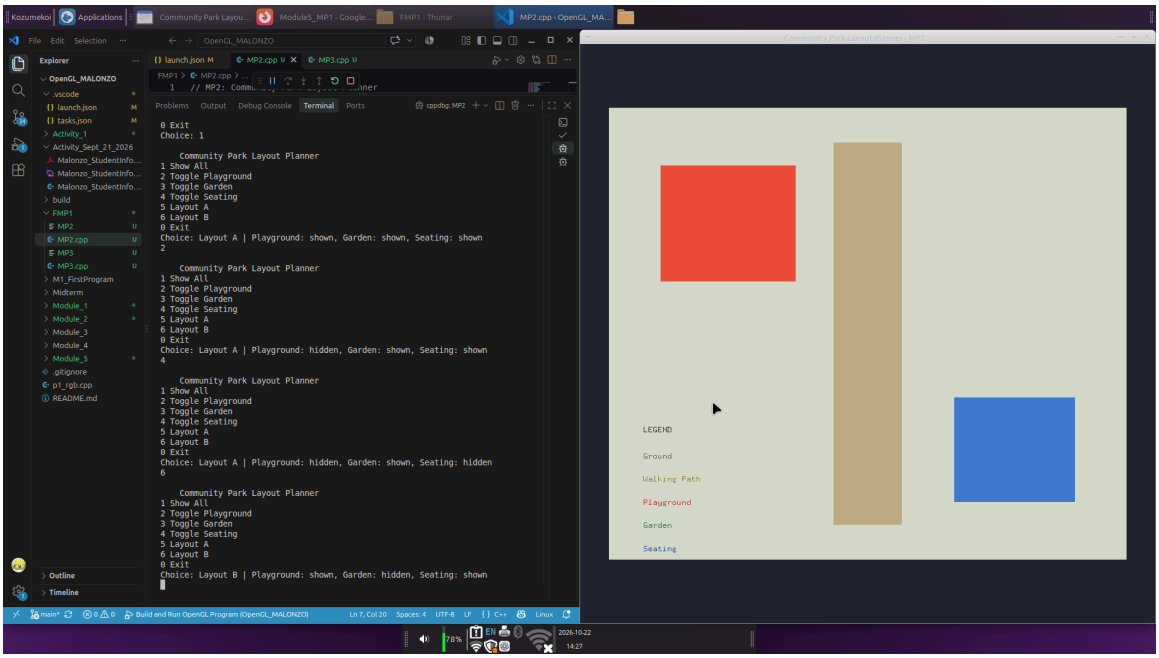
6. Draw a park boundary, walking path, and at least three distinct rectangular features (for example: playground, garden, seating area).
7. Use a GL_ARRAY_BUFFER for vertex coordinates and GL_ELEMENT_ARRAY_BUFFER for indices; demonstrate vertex sharing in at least one rectangular feature.
8. Provide menu choices: 1 Show All, 2 Toggle Playground, 3 Toggle Garden, 4 Toggle Seating, 5 Layout A, 6 Layout B, 0 Exit.
9. Accept user menu input, validate selections, and redraw the scene to reflect visibility or layout changes.
10. Include a brief on-screen legend or console legend describing each feature and its color.

Expected Output

A functioning graphical window that responds to the operator's menu selections, with clear visual changes and no crashes on invalid input.


Legend: ground=light gray-green, path=tan, playground=red, garden=green, seating=blue.
Community Park Layout Planner
1 Show All
2 Toggle Playground
3 Toggle Garden
4 Toggle Seating
5 Layout A
6 Layout B
0 Exit
Choice: 1
Community Park Layout Planner
1 Show All
2 Toggle Playground
3 Toggle Garden
4 Toggle Seating
5 Layout A
6 Layout B
0 Exit
Choice: Layout A | Playground: shown, Garden: shown, Seating: shown
Community Park Layout Planner - MP2
LEGEND
Ground
Walking Path
Playground
Garden
Seating


Legend: ground=light gray-green, path=tan, playground=red, garden=green, seating=blue.
Community Park Layout Planner
1 Show All
2 Toggle Playground
3 Toggle Garden
4 Toggle Seating
5 Layout A
6 Layout B
0 Exit
Choice: 1
Community Park Layout Planner
1 Show All
2 Toggle Playground
3 Toggle Garden
4 Toggle Seating
5 Layout A
6 Layout B
0 Exit
Choice: Layout A | Playground: shown, Garden: shown, Seating: shown
2
Community Park Layout Planner
1 Show All
2 Toggle Playground
3 Toggle Garden
4 Toggle Seating
5 Layout A
6 Layout B
0 Exit
Choice: Layout A | Playground: hidden, Garden: shown, Seating: shown
4
Community Park Layout Planner
1 Show All
2 Toggle Playground
3 Toggle Garden
4 Toggle Seating
5 Layout A
6 Layout B
0 Exit
Choice: Layout A | Playground: hidden, Garden: shown, Seating: hidden
LEGEND
Ground
Walking Path
Playground
Garden
Seating




```
    Community Park Layout Planner
1 Show All
2 Toggle Playground
3 Toggle Garden
4 Toggle Seating
5 Layout A
6 Layout B
0 Exit
Choice: Layout B | Playground: shown, Garden: hidden, Seating: shown
0
[1] + Done                       "/usr/bin/gdb" --interpreter=mi --tty=${DbgTer
m} 0<"/tmp/Microsoft-MIEngine-In-jwbo3tmc.ad2" 1>"/tmp/Microsoft-MIEngine-Out-f
eyfn03i.n5p"
kozumekoi@kozumekoi-NBLK-WAX9X:~/Documents/Codex/2026-09-14/OpenGL_Malonzo/Open
GL MALONZO$
```

Open-Ended Questions

1. Why can four unique vertices and six indices be preferable to six duplicated vertex positions for a rectangle?

Using four unique vertices and six indices saves memory by not repeating the shared corner positions of the two triangles. It also helps the GPU run faster because it can store the calculated vertices in its cache and reuse them instead of recalculating duplicate points. In the provided code, this keeps the layout clean and organized so each rectangle feature can easily be drawn using a consistent index offset.

2. What is the difference between GL_ARRAY_BUFFER and GL_ELEMENT_ARRAY_BUFFER?

`GL_ARRAY_BUFFER` is used to store vertex attributes like coordinates, which define where the shapes are located in 2D space. On the other hand, `GL_ELEMENT_ARRAY_BUFFER` stores index data that tells OpenGL which order to connect

those vertices to draw triangles. In short, the array buffer holds the raw vertex points while the element array buffer holds the instructions for combining them.

3. How would you extend the program to support user-defined park dimensions while keeping indices valid?

To let users customize dimensions, you can keep the original index array unchanged since the way vertices connect into rectangles stays the same. You would update the vertex buffer initialization from `GL_STATIC_DRAW` to `GL_DYNAMIC_DRAW` so the vertex coordinates can change at runtime. Then, whenever the user enters new dimensions, you recalculate the vertex positions in C++ and upload them to the GPU using `glBufferSubData`.

4. What are the trade-offs between rebuilding buffers for each layout and maintaining multiple preloaded layouts?

Rebuilding buffers on the fly saves GPU memory and gives you infinite freedom to handle any user-defined dimensions, but it adds a small performance cost from updating data repeatedly. Preloading multiple layouts into memory makes switching layouts instant and smooth without CPU-to-GPU transfer delays. However, preloading uses more GPU memory and restricts the program to only a few fixed, hardcoded designs.

Submission Checklist

/☐ C++ source file(s) /☐ Output screenshots /☐ Menu/input evidence ☐ Written answers

MACHINE PROBLEM 03

Warehouse Robot Route Simulator

Advanced • Dynamic VBO + glMapBuffer()

Scenario

A warehouse supervisor wants to demonstrate how a robot icon moves between storage stations. The prototype should allow operators to change the robot position and speed and observe the movement on a simple 2D warehouse map.

Problem Statement

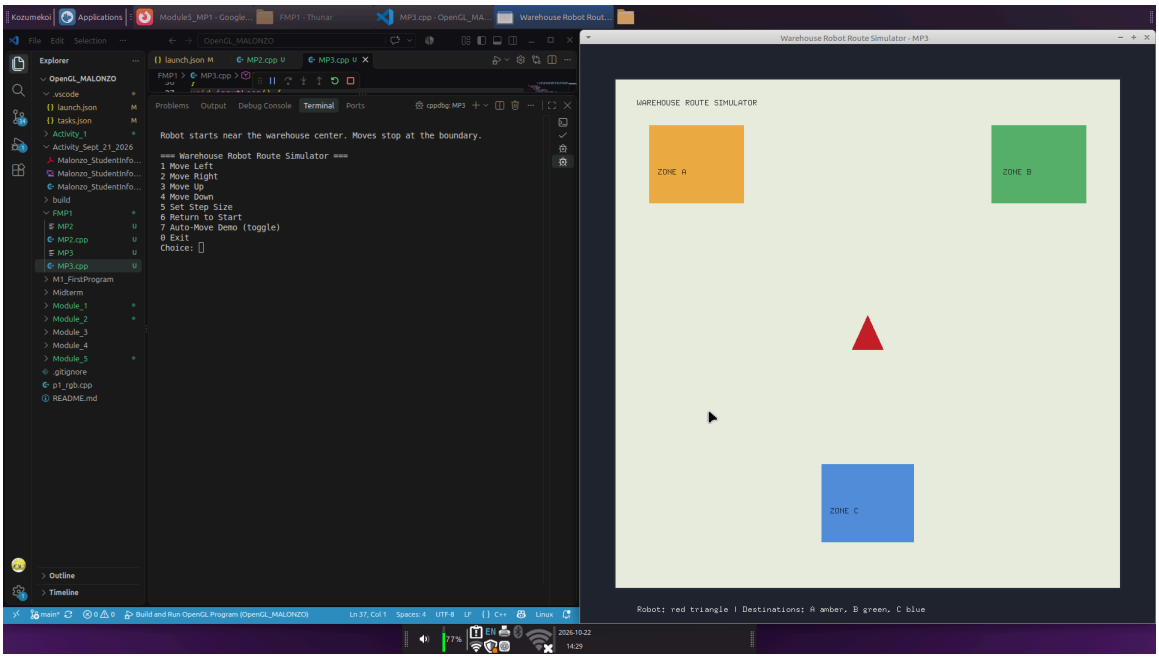
Build a menu-driven OpenGL simulator with a robot represented by a triangle and three labeled or color-coded destination zones. Store the robot vertex positions in a VBO. Modify the robot geometry through glMapBuffer() and glUnmapBuffer() (or a documented safe buffer-update fallback if mapping is unavailable), then redraw after each movement step.

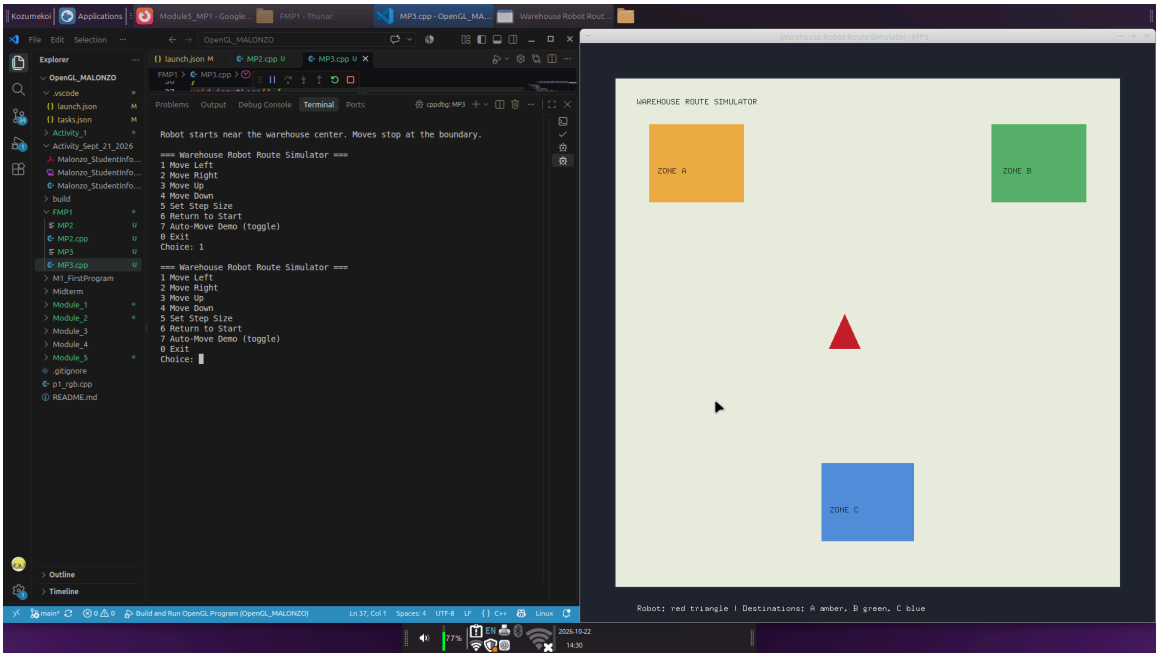
Functional and Technical Requirements

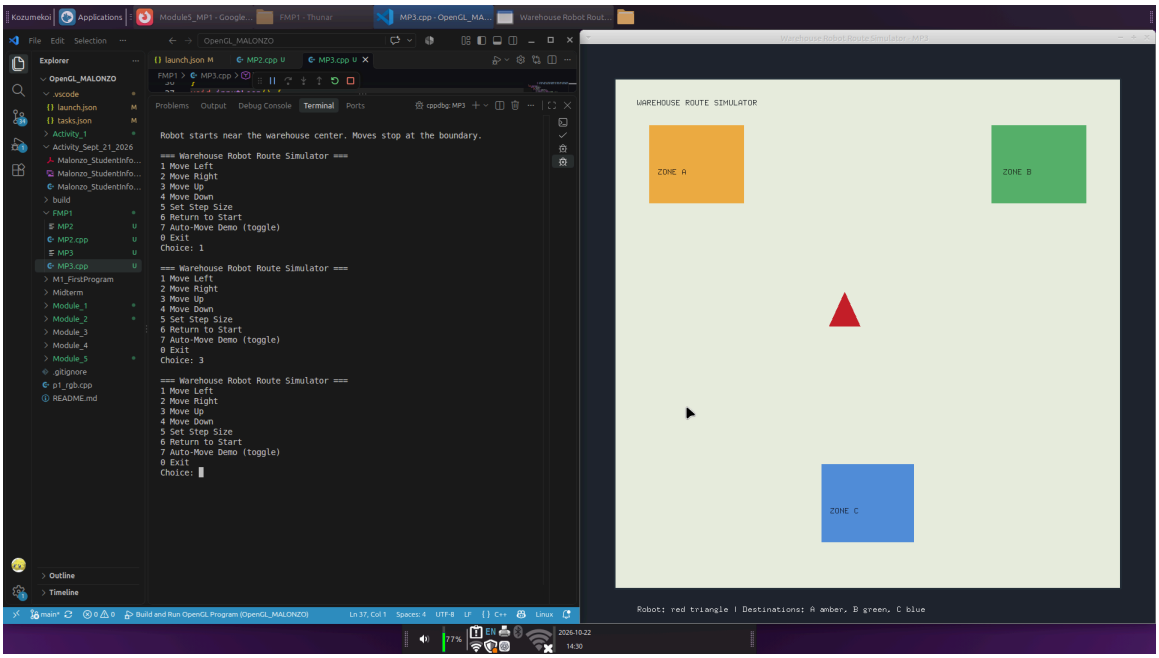
11. Draw a warehouse boundary, three destination zones, and a triangle representing the robot.
12. Create the robot VBO once and use GL_DYNAMIC_DRAW as the intended update hint.
13. Provide menu choices: 1 Move Left, 2 Move Right, 3 Move Up, 4 Move Down, 5 Set Step Size, 6 Return to Start, 7 Auto-Move Demo, 0 Exit.
14. Read numeric menu choices and a positive step size from the user; reject invalid entries and prevent the robot from leaving the warehouse boundary.
15. For movement, modify the correct X or Y components of all robot vertices using mapped buffer data; check mapping success and unmap after updates.
16. For Auto-Move Demo, use a timer callback and request redisplay; do not schedule a new timer from every display callback.

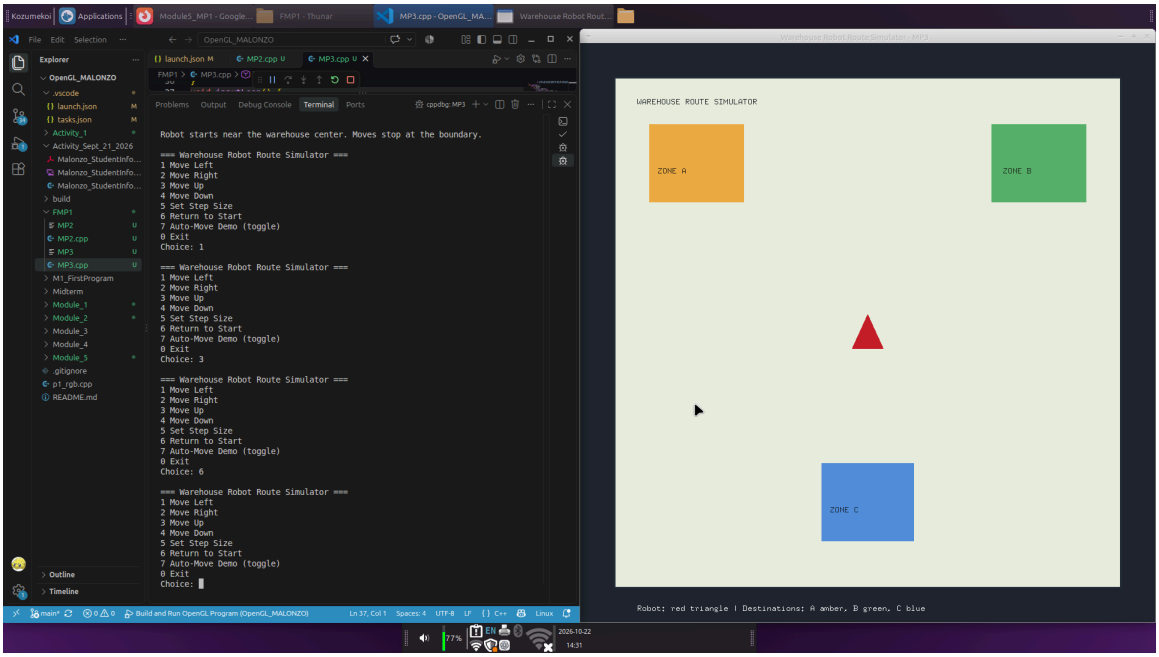
Expected Output

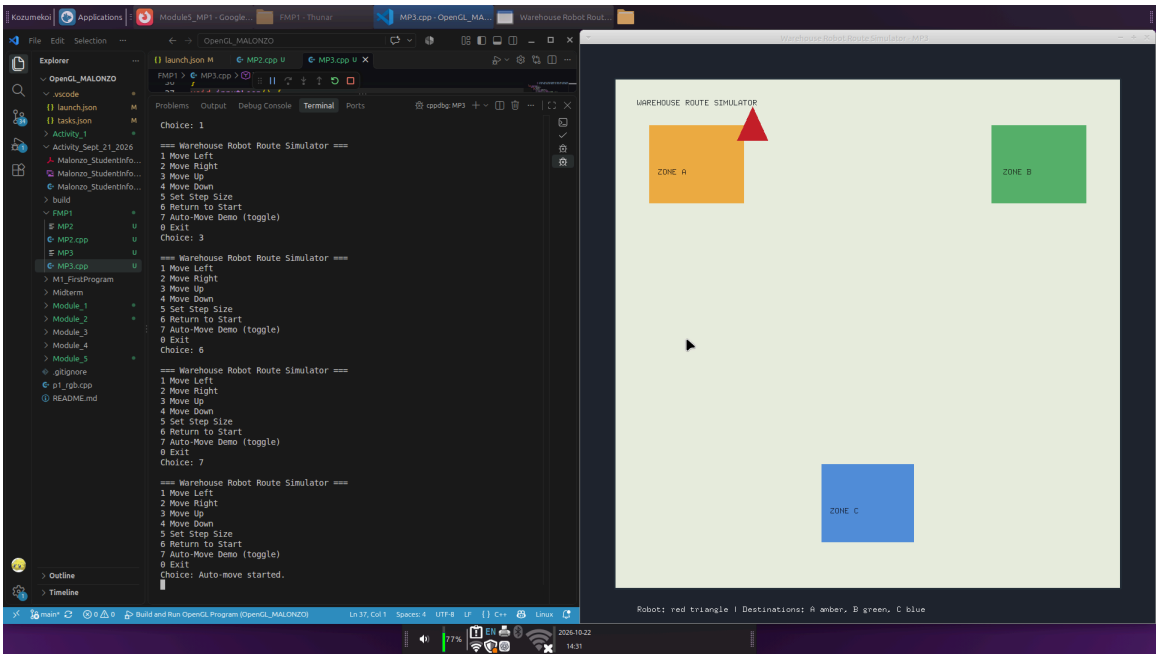
A functioning graphical window that responds to the operator’s menu selections, with clear visual changes and no crashes on invalid input.

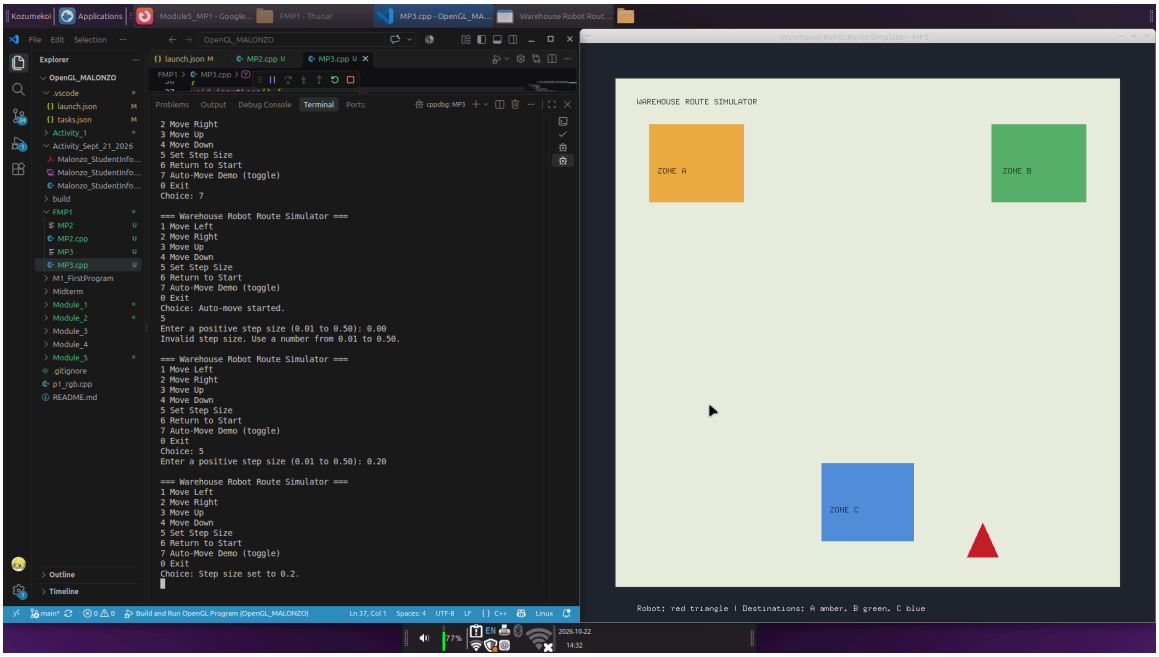

Robot starts near the warehouse center. Moves stop at the boundary.
=== Warehouse Robot Route Simulator ===
1 Move Left
2 Move Right
3 Move Up
4 Move Down
5 Set Step Size
6 Return to Start
7 Auto-Move Demo (toggle)
0 Exit
Choice:
WAREHOUSE ROUTE SIMULATOR
ZONE A
ZONE B
ZONE C
Robot: red triangle | Destinations: A amber, B green, C blue


Robot starts near the warehouse center. Moves stop at the boundary.
=== Warehouse Robot Route Simulator ===
1 Move Left
2 Move Right
3 Move Up
4 Move Down
5 Set Step Size
6 Return to Start
7 Auto-Move Demo (toggle)
0 Exit
Choice: 1
=== Warehouse Robot Route Simulator ===
1 Move Left
2 Move Right
3 Move Up
4 Move Down
5 Set Step Size
6 Return to Start
7 Auto-Move Demo (toggle)
0 Exit
Choice:
WAREHOUSE ROUTE SIMULATOR
ZONE A
ZONE B
ZONE C
Robot: red triangle | Destinations: A amber, B green, C blue


Robot starts near the warehouse center. Moves stop at the boundary.
=== Warehouse Robot Route Simulator ===
1 Move Left
2 Move Right
3 Move Up
4 Move Down
5 Set Step Size
6 Return to Start
7 Auto-Move Demo (toggle)
0 Exit
Choice: 1
=== Warehouse Robot Route Simulator ===
1 Move Left
2 Move Right
3 Move Up
4 Move Down
5 Set Step Size
6 Return to Start
7 Auto-Move Demo (toggle)
0 Exit
Choice: 3
=== Warehouse Robot Route Simulator ===
1 Move Left
2 Move Right
3 Move Up
4 Move Down
5 Set Step Size
6 Return to Start
7 Auto-Move Demo (toggle)
0 Exit
Choice:
WAREHOUSE ROUTE SIMULATOR
ZONE A
ZONE B
ZONE C
Robot: red triangle | Destinations: A amber, B green, C blue


Robot starts near the warehouse center. Moves stop at the boundary.
=== Warehouse Robot Route Simulator ===
1 Move Left
2 Move Right
3 Move Up
4 Move Down
5 Set Step Size
6 Return to Start
7 Auto-Move Demo (toggle)
0 Exit
Choice: 1
=== Warehouse Robot Route Simulator ===
1 Move Left
2 Move Right
3 Move Up
4 Move Down
5 Set Step Size
6 Return to Start
7 Auto-Move Demo (toggle)
0 Exit
Choice: 3
=== Warehouse Robot Route Simulator ===
1 Move Left
2 Move Right
3 Move Up
4 Move Down
5 Set Step Size
6 Return to Start
7 Auto-Move Demo (toggle)
0 Exit
Choice: 6
=== Warehouse Robot Route Simulator ===
1 Move Left
2 Move Right
3 Move Up
4 Move Down
5 Set Step Size
6 Return to Start
7 Auto-Move Demo (toggle)
0 Exit
Choice:
WAREHOUSE ROUTE SIMULATOR
ZONE A
ZONE B
ZONE C
Robot: red triangle | Destinations: A amber, B green, C blue





Open-Ended Questions

1. Why must all three triangle vertices change by the same displacement to move the robot without deforming it?

Moving all three vertices by the exact same displacement keeps the shape and size of the triangle intact. In the `moveRobot` function, adding `dx` and `dy` to every vertex shifts the robot as a rigid body across the screen. If the vertices were modified by different values, the distances between them would change, causing the triangle to stretch, shrink, or distort.

2. How does glMapBuffer() connect to C++ pointers and array indexing?

Calling `glMapBuffer` maps the GPU's memory buffer into the CPU's memory address space and returns a pointer. In the code, this void pointer is cast to a `Vertex*` array pointer named `mapped`, allowing C++ to directly modify the buffer. standard array indexing like `mapped[i]` can then be used to read and update the x and y coordinates of each vertex in the VBO directly.

3. What synchronization problems may occur if the CPU updates a buffer while the GPU is using it?

If the CPU tries to write to the vertex buffer while the GPU is reading it to render a frame, a data race occurs. This can cause rendering artifacts, such as flickering or rendering a half-updated robot with misplaced vertices. In severe cases, driver-level synchronization conflicts could crash the graphics pipeline or result in buffer corruption.

4. When would a transformation matrix be a better choice than modifying vertex coordinates directly?

A transformation matrix is better when handling complex operations like rotation, scaling, or multiple hierarchical objects. Direct vertex modifications force the CPU to loop through every point and update VBO memory repeatedly using `glMapBuffer`. Using matrices offloads the math entirely to the GPU vertex shader, making it much faster and cleaner for complex movements.

5. How would you improve this simulator to support multiple robots while minimizing buffer updates?

To support multiple robots efficiently, all robot vertices should be combined into a single, larger VBO instead of mapping buffers for individual movements. A dynamic uniform buffer or transform matrix can then be passed to the GPU shader to update positions instantly without modifying vertex data. This keeps buffer updates to a minimum and reduces expensive CPU-to-GPU memory operations.

Submission Checklist

/☐ C++ source file(s) /☐ Output screenshots /☐ Menu/input evidence ☐ Written answers